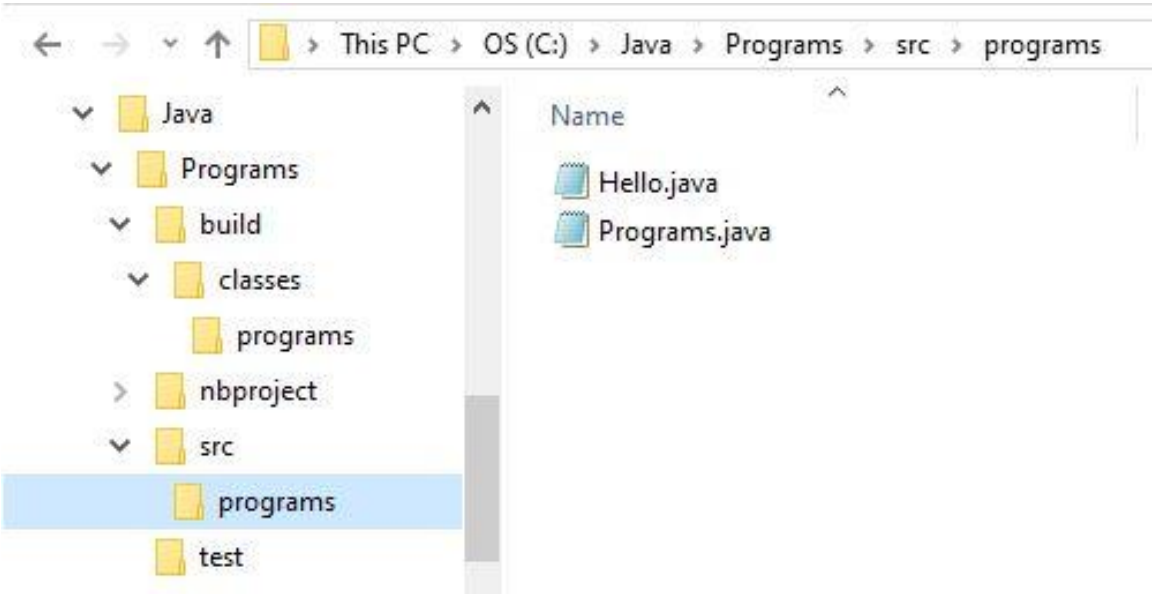
How to run Java programs from the command prompt

This handout will be using NetBeans and assuming you have created a project with ANT. Any other project type created will not follow this handout. This handout will run java program called `Hello.java`.

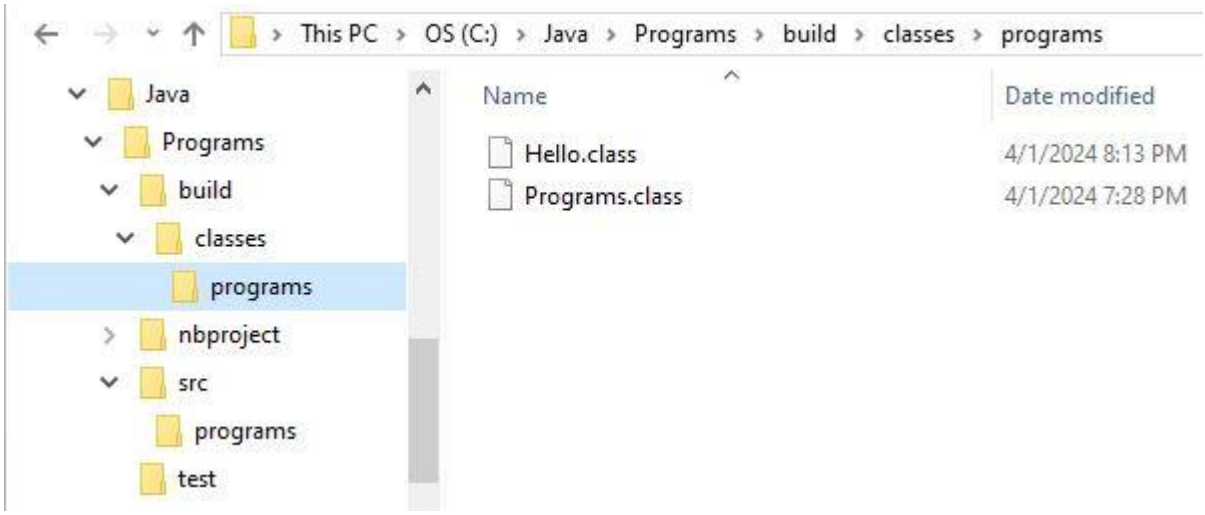
This handout will also be using the following:

C:	will be the hard drive used
Java	will be folder on drive C: that contains the java project
Programs	name of the NetBeans project, made by ANT
Hello.java	sample java program to run

Picture below shows the tree structure for this project with the files found in this project. These java files are always under folder "**`src`**" in subfolder with project name.



Once you build a java program and it compiles successfully, then the java compiler/linker creates file name that ends in **`.class`**, and is saved under the project name folder, under the **`classes`** folder under the **`build`** folder. See picture below for tree structure, **`Hello.class`** is this file.



This file, the one that ends in **`.class`**, is the file that you want to run from the command line. Next you will then need to open up the command window and navigate to the desired location. Before you do this, review below for some basic commands to use.

Some basic DOS/Windows commands

`cd FolderName`	`cd` is change directory, `FolderName` is name of folder to change to
`cd..`	will go back or up one level
`dir`	does a listing of the current working folder/directory
`type FolderName`	will display contents of this folder to the screen

How to open command window

To open up the command window, type in the search bar, `command prompt`, then click on the command prompt icon. Window similar to one below will appear. Note that by default the command prompt window will be black background with light grey letters. I changed format for printed pages not to use lots of toner.

```
Command Prompt
Microsoft Windows [Version 10.0.19045.4170]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Matt>
```

Now using the above DOS/Windows commands, navigate to your **`classes`** folder, see picture. Note this path structure corresponds to my system and yours will be a bit different depending on where your java project is located.

```
Command Prompt
Microsoft Windows [Version 10.0.19045.4170]
(c) Microsoft Corporation. All rights reserved.

C:\Users\Matt>cd..

C:\Users>cd..

C:\>cd java

C:\Java>cd programs

C:\Java\Programs>cd build

C:\Java\Programs\build>cd classes

C:\Java\Programs\build\classes>
```

The above picture shows the commands I used to get to the correct folder. All this is really important is to have the following line:

```
C:\Java\Programs\build\classes>
```

To Run Java Program from command line

When you want to run the java class, for this example we will be running `Hello.class`, you need to be in the folder `classes`, and the syntax is then:

```
C:\Java\Programs\build\classes>java PackageName.FileName
```

Remember that java is case sensitive, so the spelling is very important with either upper or lower case letters. To run the Hello file type information and then press enter, output shown below.
(Note: The package name is often the same as the project name.)

```
C:\Java\Programs\build\classes>java programs.Hello
Hello Java World

C:\Java\Programs\build\classes>
```

Using I/O Redirection at the command line

You can have the input come from another file, but where should this file be saved if want to redirect the input from the command line use the "**<**" input redirection operator? These files, which will be used for input, should be stored in the **classes** folder. They are not to be in the same folder as the **.class** files. To read from a file at the command prompt, syntax is given below, with same path have been using:

```
C:\Java\Programs\build\classes>java PackageName.FileName < InputName
```

Output can also be redirected, by using the "**>**" output redirection operator, and when done so the file will be created and stored in the classes folder, or the same location as the input files are stored. See example below:

```
C:\Java\Programs\build\classes>java PackageName.FileName > OutputName
```

A program can also do both input and output redirection, see below. Note the whole path was not displayed this time.

```
classes>java PackageName.FileName < InputName > OutputName
```